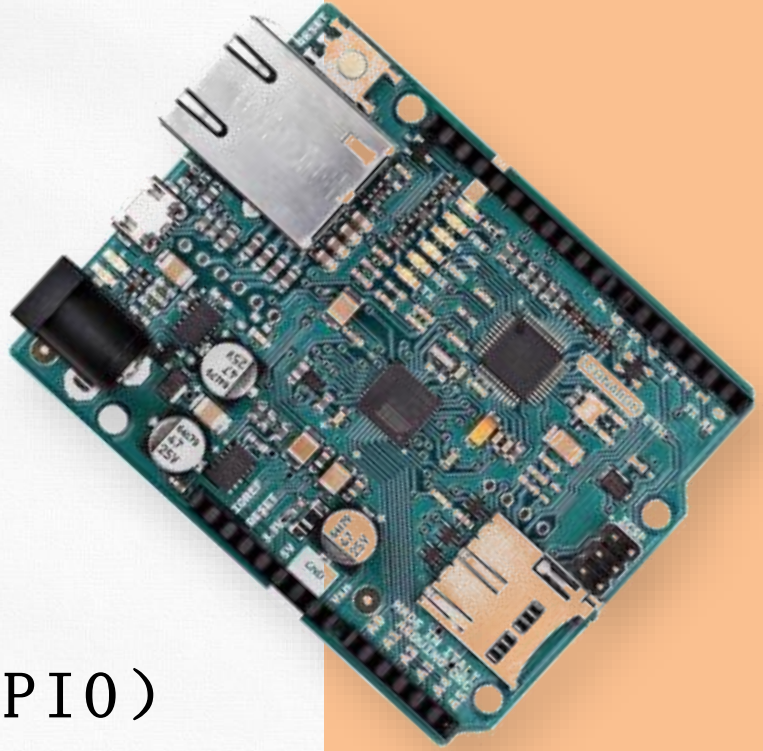
单片机



第4章：通用输入输出接口（GPIO）

目 录

CONTENTS

01.GPIO概述

02.STM32的GPIO工作原理

03.GPIO模块的HAL库接口函数及应用

01

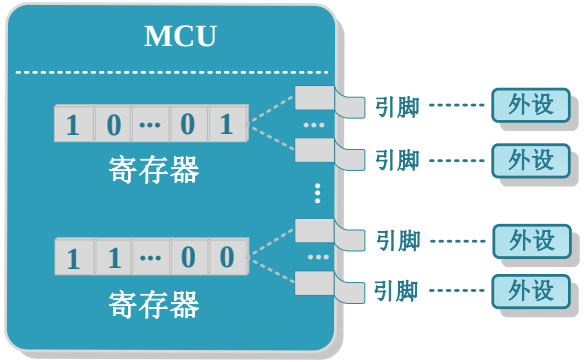
概　述

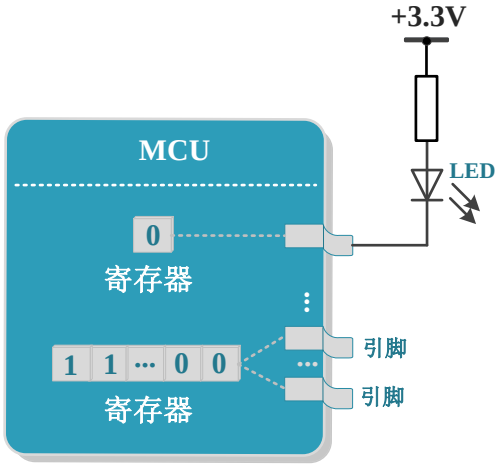
GPIO概述

GPIO（General Purpose Input/Output，通用输入/输出）是微控制器最基本的片上外设，微控制器通过GPIO实现与外界的信息交换。MCU的I/O引脚既可以感知外界的信息/信号（I/O 引脚的输入模式），又可以操控某个外部设备（ I/O 引脚的输出模式）。

GPIO概述

GPIO端口内部是由一个个寄存器组成，一个引脚对应一个寄存器或多个引脚对应一个多位的寄存器，改变寄存器中的数据就可以改变外设的工作方式。





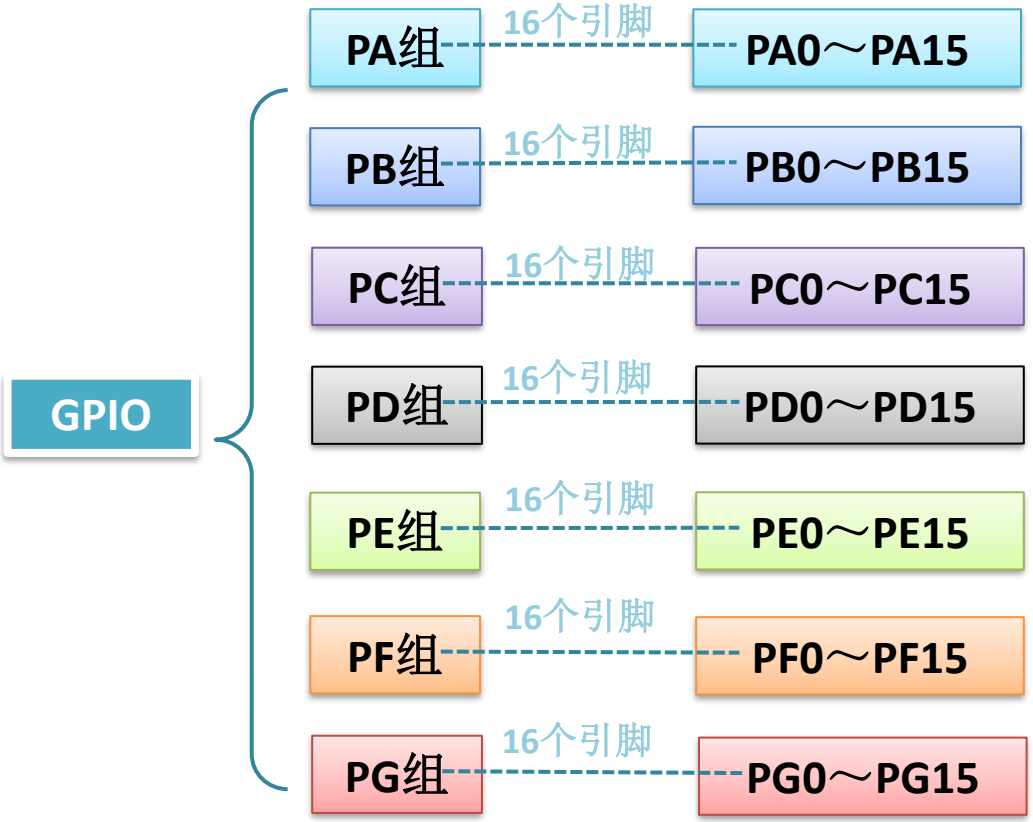
GPIO概述

端口（PORT）：

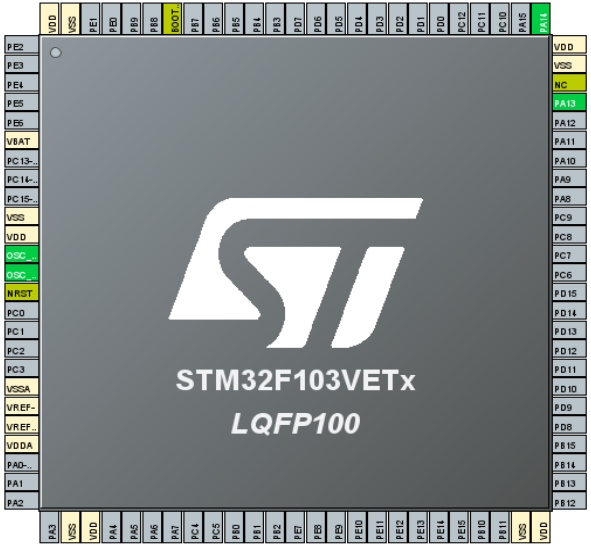
STM32微控制器将GPIO分成了很多个组，每一组称为一个端口，如GPIOA、GPIOB、GPIOC

引脚（PIN）：

引脚对应微控制器的一个管脚，归属于端口，每一组端口最多包括16个引脚。如端口GPIOA对应的16个引脚为PA0~PA15



GPIO概述



STM32F103系列微控制器的型号不同，GPIO引脚数量也不相同。一般可将引脚分为六大类：

（1）电源引脚：V_{DD}、V_{SS}、V_{REF+}、V_{REF-}、V_{DDA}、V_{SSA}、V_{BAT}；

（2）晶振引脚：OSC_IN、OSC_OUT等；

（3）复位引脚：NRST；

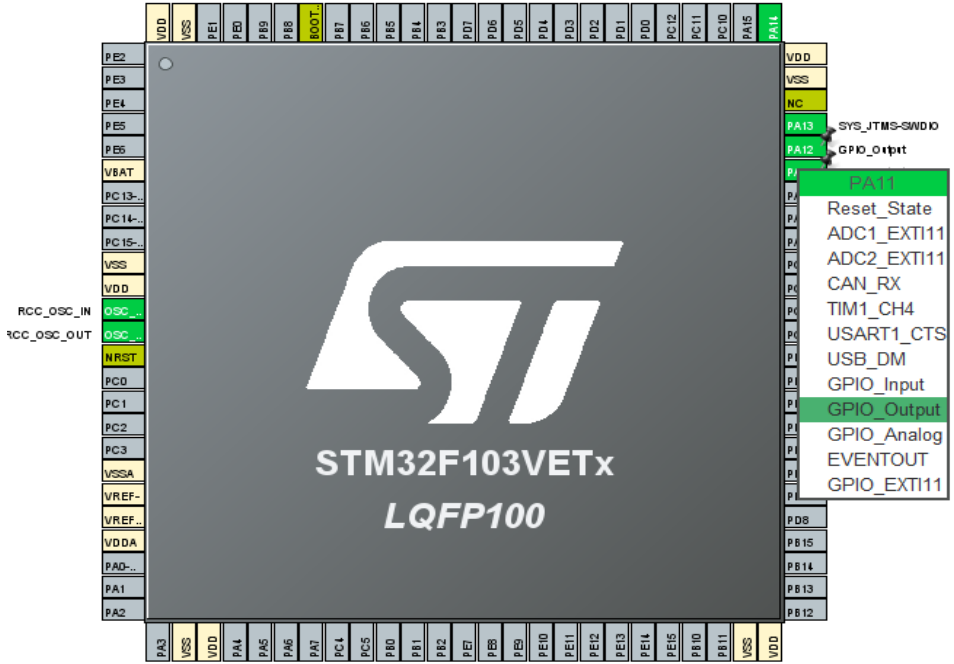
（4）BOOT引脚：BOOT0、BOOT1；

（5）程序下载引脚：JTAG、SWD、DAP等下载方式

（6）GPIO引脚：最多可有7组GPIO，分别为：

PA组：PA0~PA15；PB组：PB0~ PB15；
PC组：PC0~ PC15；PD组：PD0~ PD15；
PE组：PE0~PE15；
PF组：PF0~PF15；PG组：PG0~PG15；(ZET6型号有)

GPIO概述

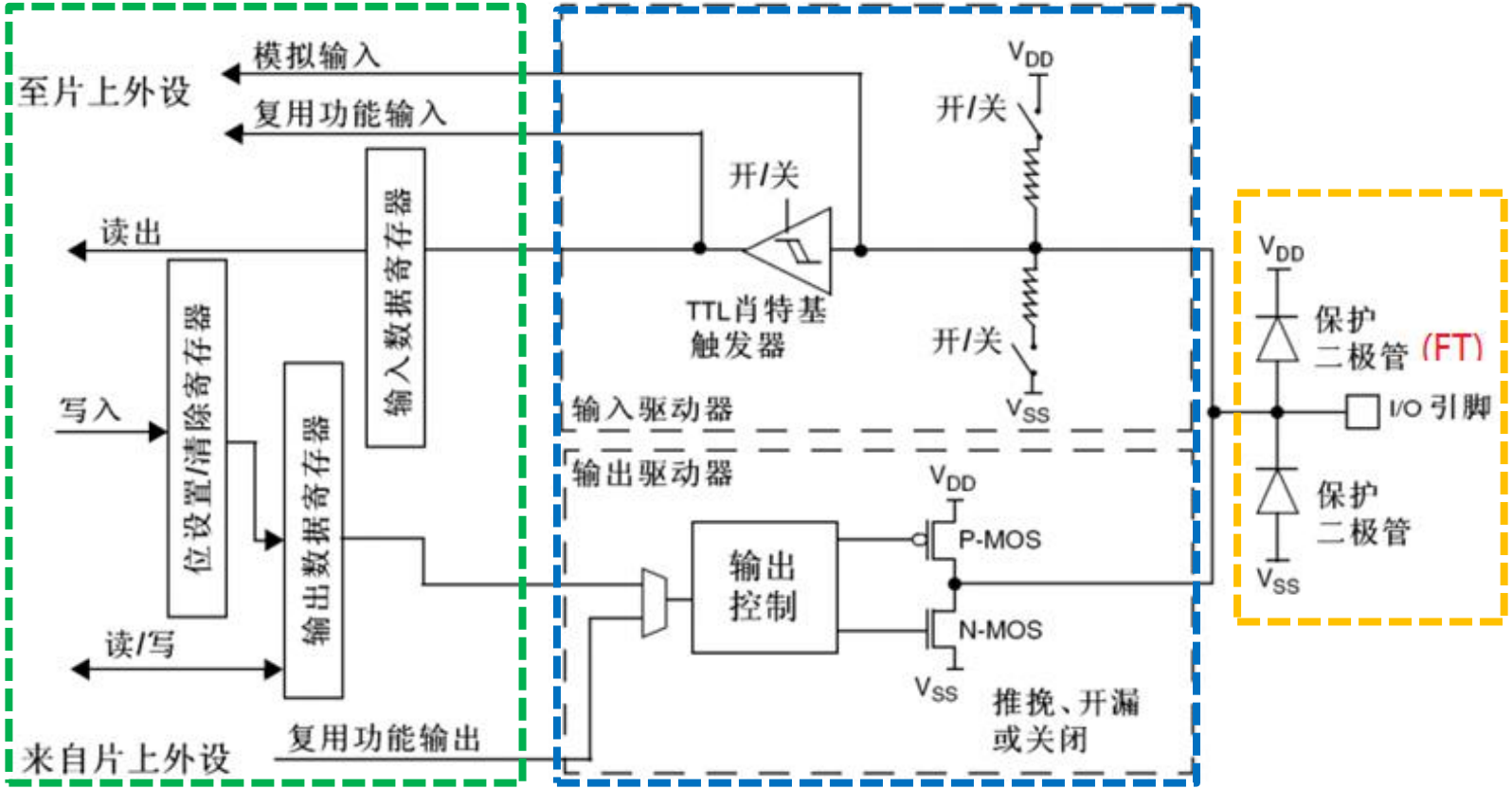


GPIO引脚除了作为通用输入/输出引脚使用外，还可以作为片上外设（如串口、ADC、IIC等）的I/O引脚，即一个引脚可以作为多个外设引脚使用，称为复用I/O端口AFIO（Alternate Function I/O），但一个引脚某一时刻只能使用复用功能中的一个。

02

STM32的GPIO工作原理

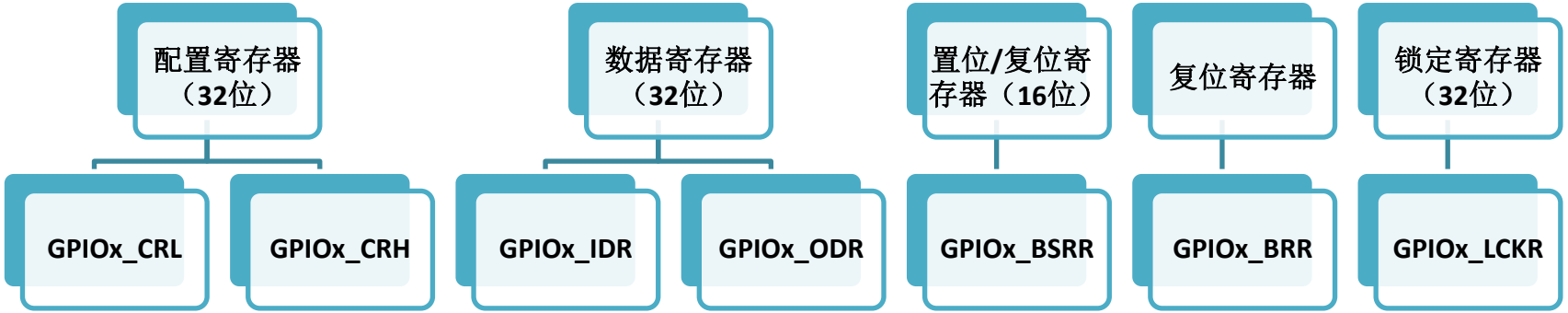
GPIO内部结构



端口寄存器组　　输入/输出驱动器　　引脚电路

GPIO内部结构

（了解）



每组GPIO端口(Px)都由7个寄存器组成，负责控制该端口的16个引脚Px0 ~ Px15。

GPIO端口的每个位可由软件分别配置成多种模式。每个I/O端口位能自由编程，I/O端口寄存器必须按32位字被访问。

（1）配置寄存器：配置GPIO的模式及状态，如输入/输出模式、复用功能及输出的最大速度等，如端口配置低寄存器（GPIOx_CRL）、端口配置高寄存器（GPIO_CRH）；

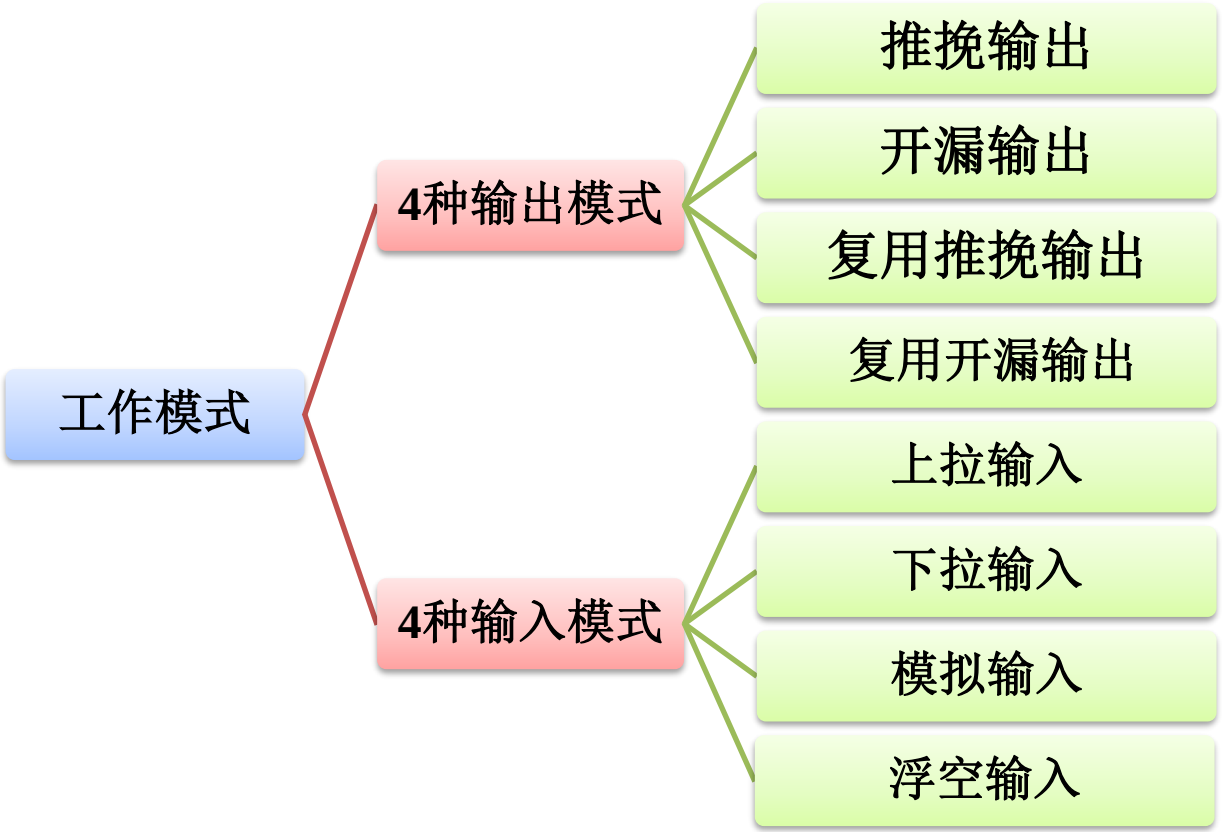
（2）数据寄存器：保存了 GPIO 的输入电平或将要输出的电平，如端口输入数据寄存器（GPIOx_IDR）、端口输出数据寄存器（GPIOx_ODR）；

（3）位控制寄存器：设置某引脚的数据 为1 或 0，如端口位设置/清除寄存器（GPIOx_BSRR）、端口位清除寄存器（GPIOx_BRR）；

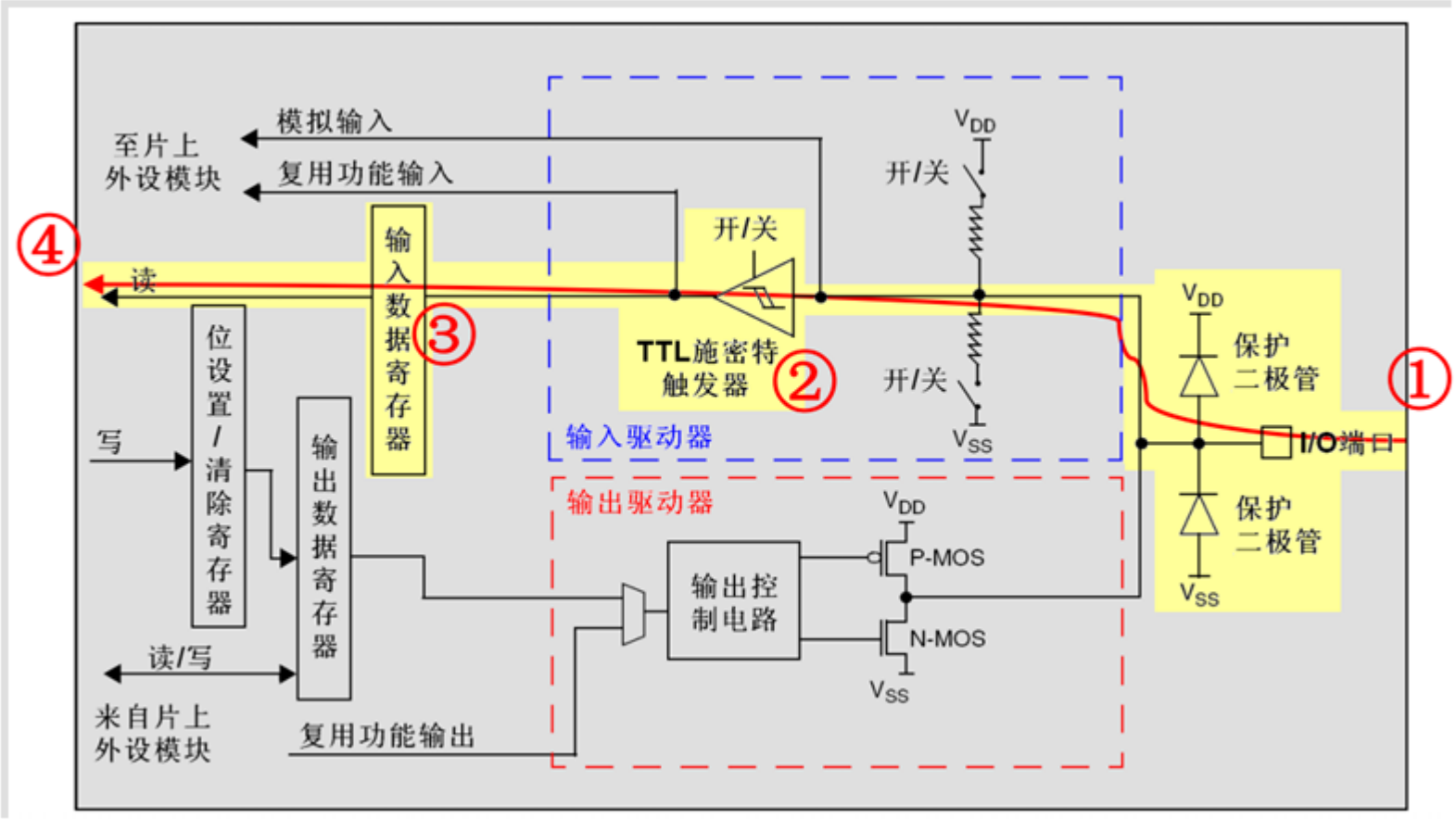
（4）锁定寄存器：设置某锁定引脚后，就不能修改其配置，如端口配置锁定寄存器（GPIOx_LCKR）（其中，x=A...E）。

GPIO端口的每位可由软件分别配置成多种模式：输入浮空、输入上拉或下拉、模拟输入、开漏输出、推挽式输出等。

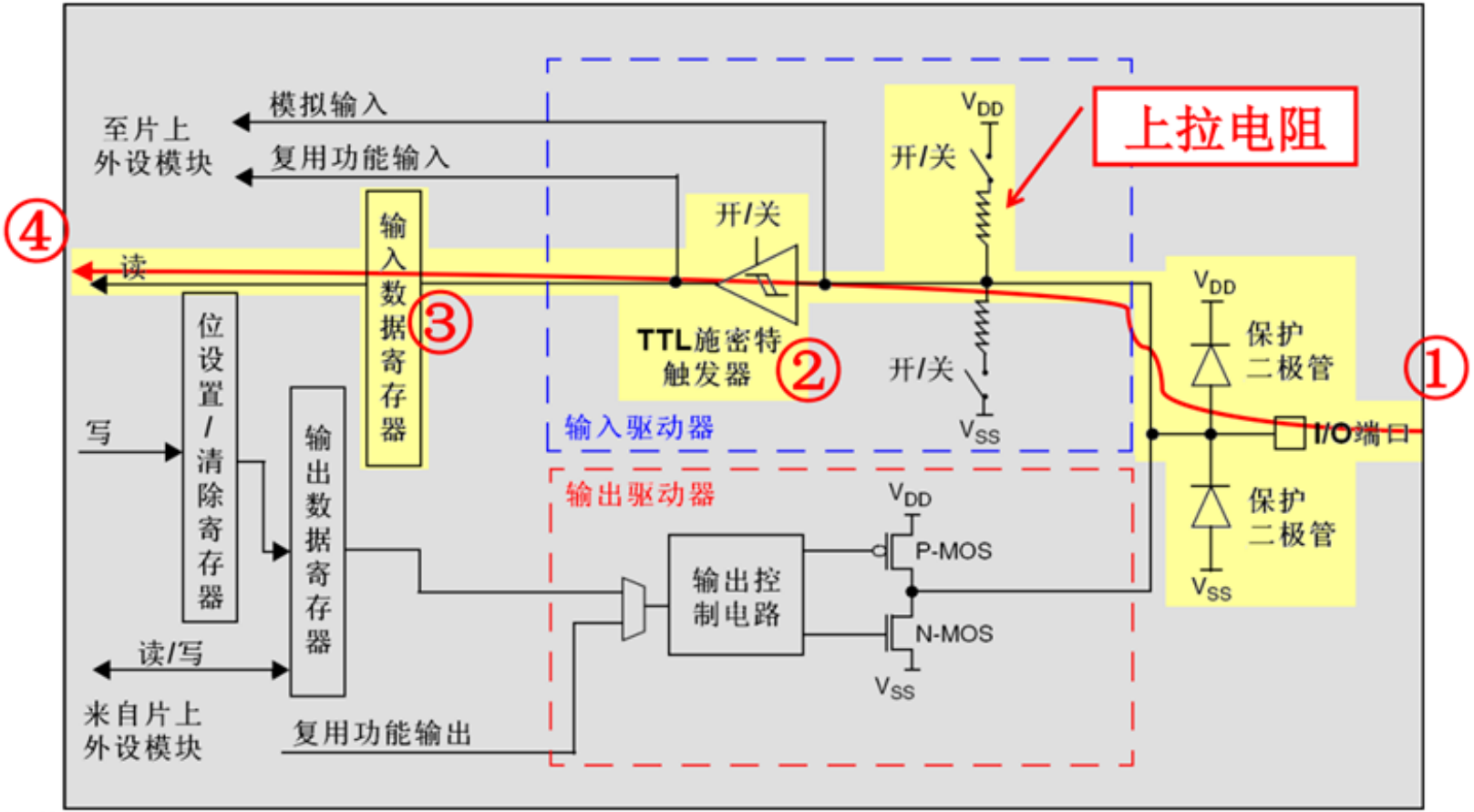
GPIO工作模式


工作模式
4种输出模式
推挽输出
开漏输出
复用推挽输出
复用开漏输出
4种输入模式
上拉输入
下拉输入
模拟输入
浮空输入

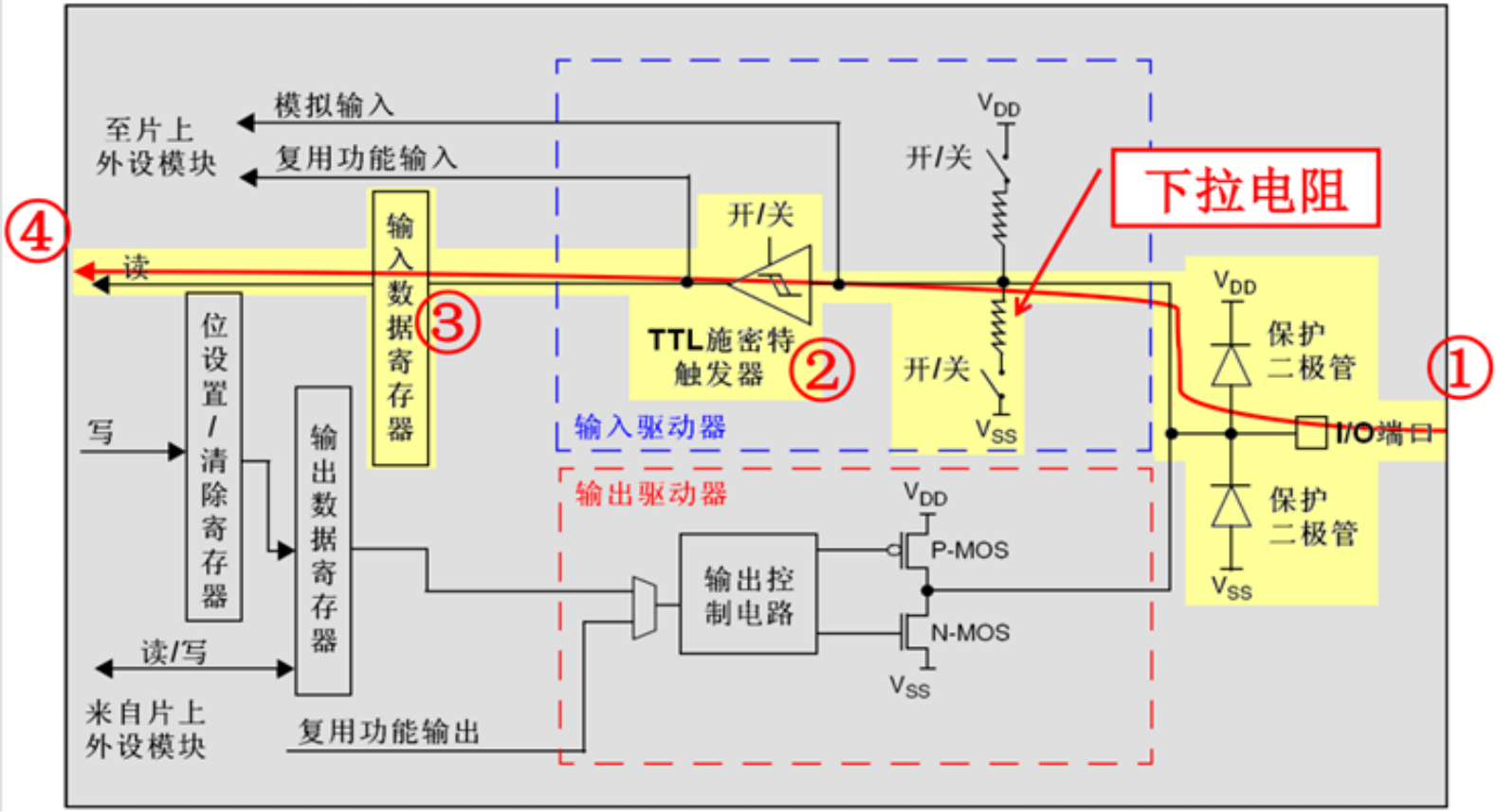
浮空输入模式


模拟输入
至片上
外设模块
复用功能输入
④
读
输入数据寄存器
③
开/关
TTL施密特
触发器
②
开/关
开/关
V_{DD}
V_{SS}
输入驱动器
V_{DD}
保护
二极管
①
I/O端口
保护
二极管
V_{SS}
写
位设置/清除寄存器
输出数据寄存器
输出驱动器
V_{DD}
P-MOS
输出控制电路
N-MOS
V_{SS}
读/写
来自片上
外设模块
复用功能输出

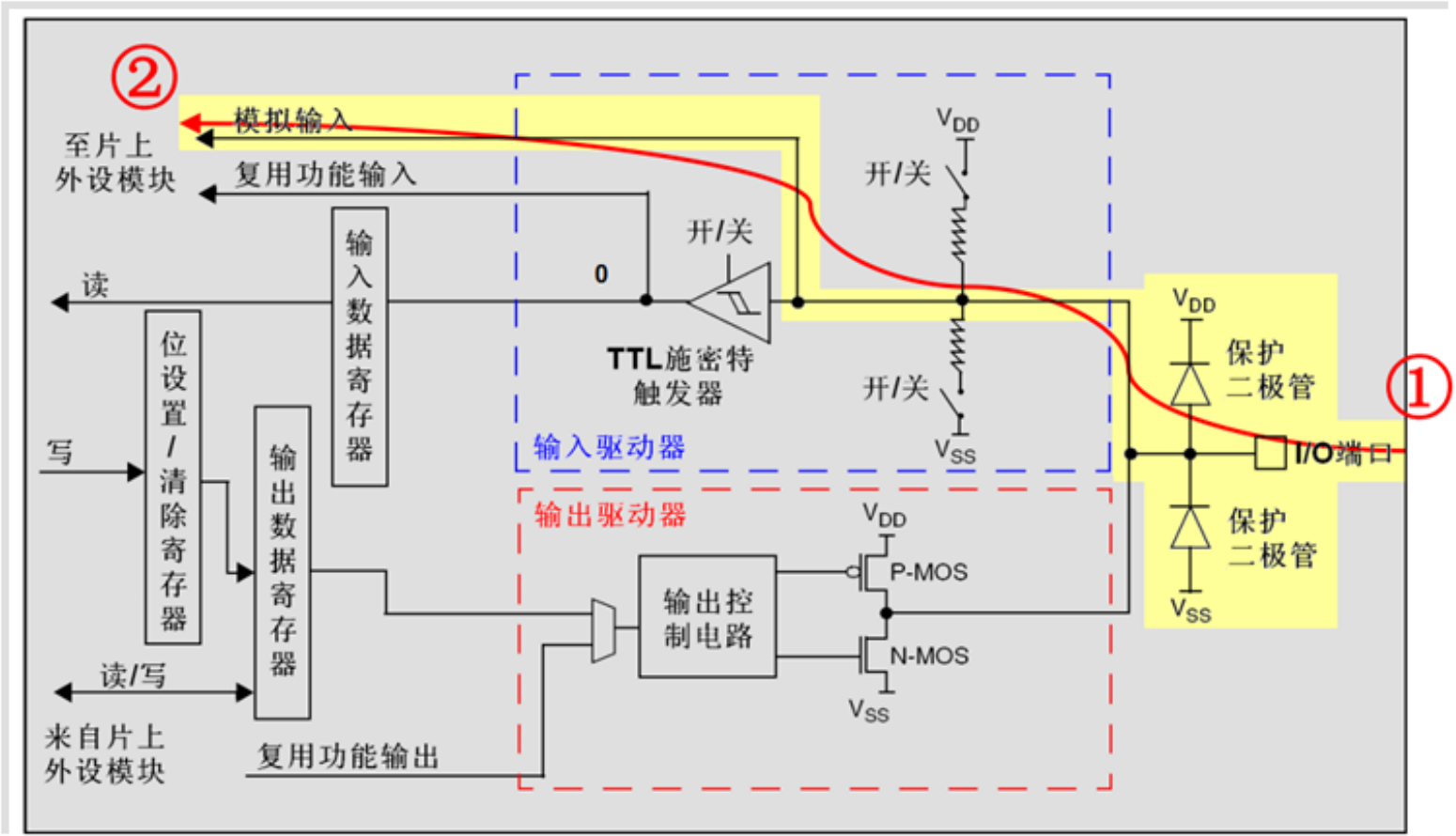
上拉输入模式


至片上
外设模块
模拟输入
复用功能输入
④
读
输入数据寄存器
③
开/关
TTL施密特
触发器
②
开/关
V_{DD}
开/关
V_{SS}
上拉电阻
输入驱动器
V_{DD}
保护
二极管
①
I/O端口
保护
二极管
V_{SS}
写
位设置/清除寄存器
输出数据寄存器
输出驱动器
V_{DD}
P-MOS
输出控制电路
N-MOS
V_{SS}
读/写
来自片上
外设模块
复用功能输出

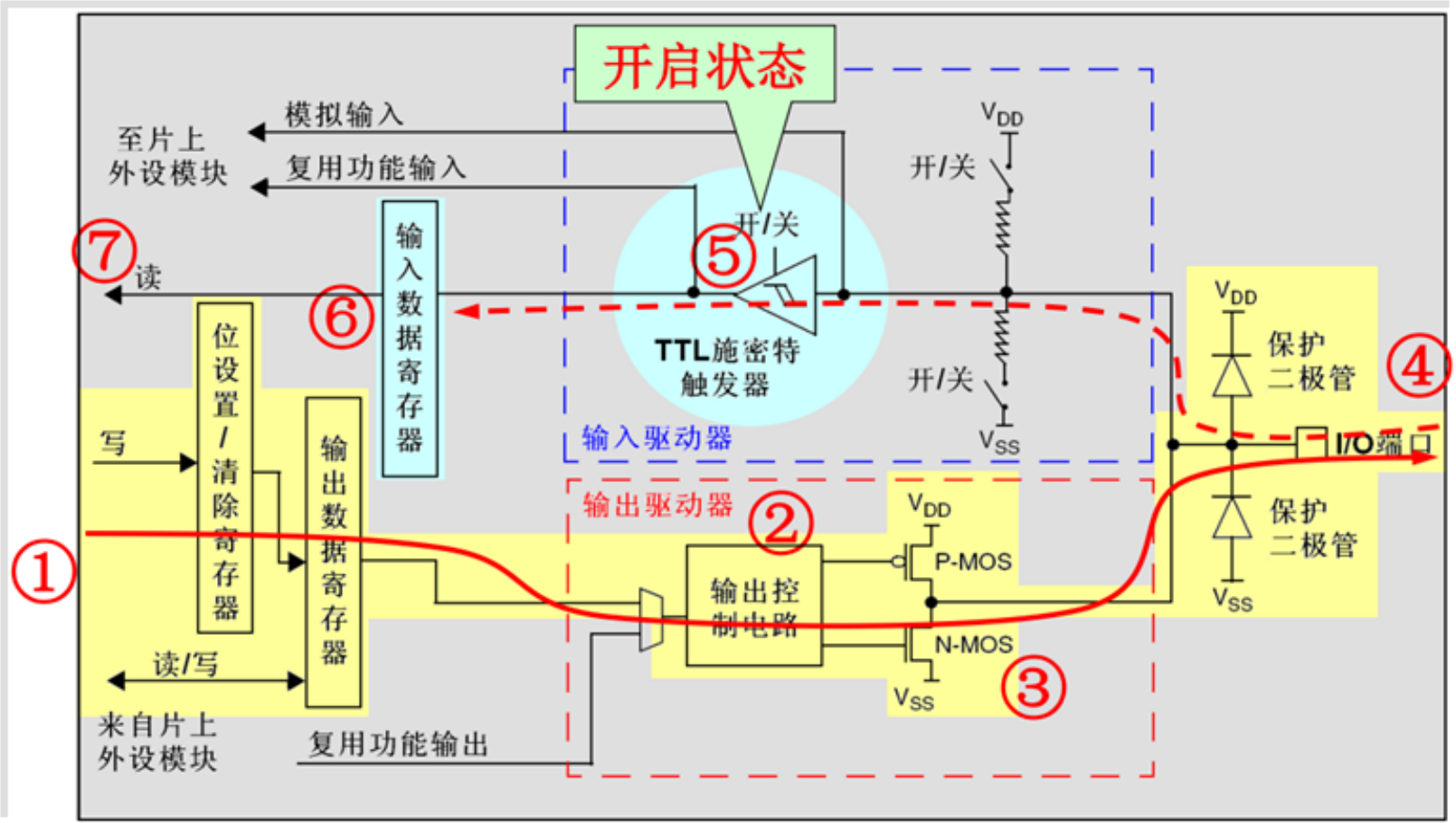
下拉输入模式


模拟输入
至片上
外设模块
复用功能输入
④
读
输入数据寄存器
③
开/关
TTL施密特
触发器
②
开/关
V_{DD}
开/关
下拉电阻
V_{DD}
保护
二极管
①
I/O端口
保护
二极管
V_{SS}
输入驱动器
输出驱动器
V_{DD}
P-MOS
输出控
制电路
N-MOS
V_{SS}
写
位设置/清除寄存器
输出数据寄存器
读/写
来自片上
外设模块
复用功能输出

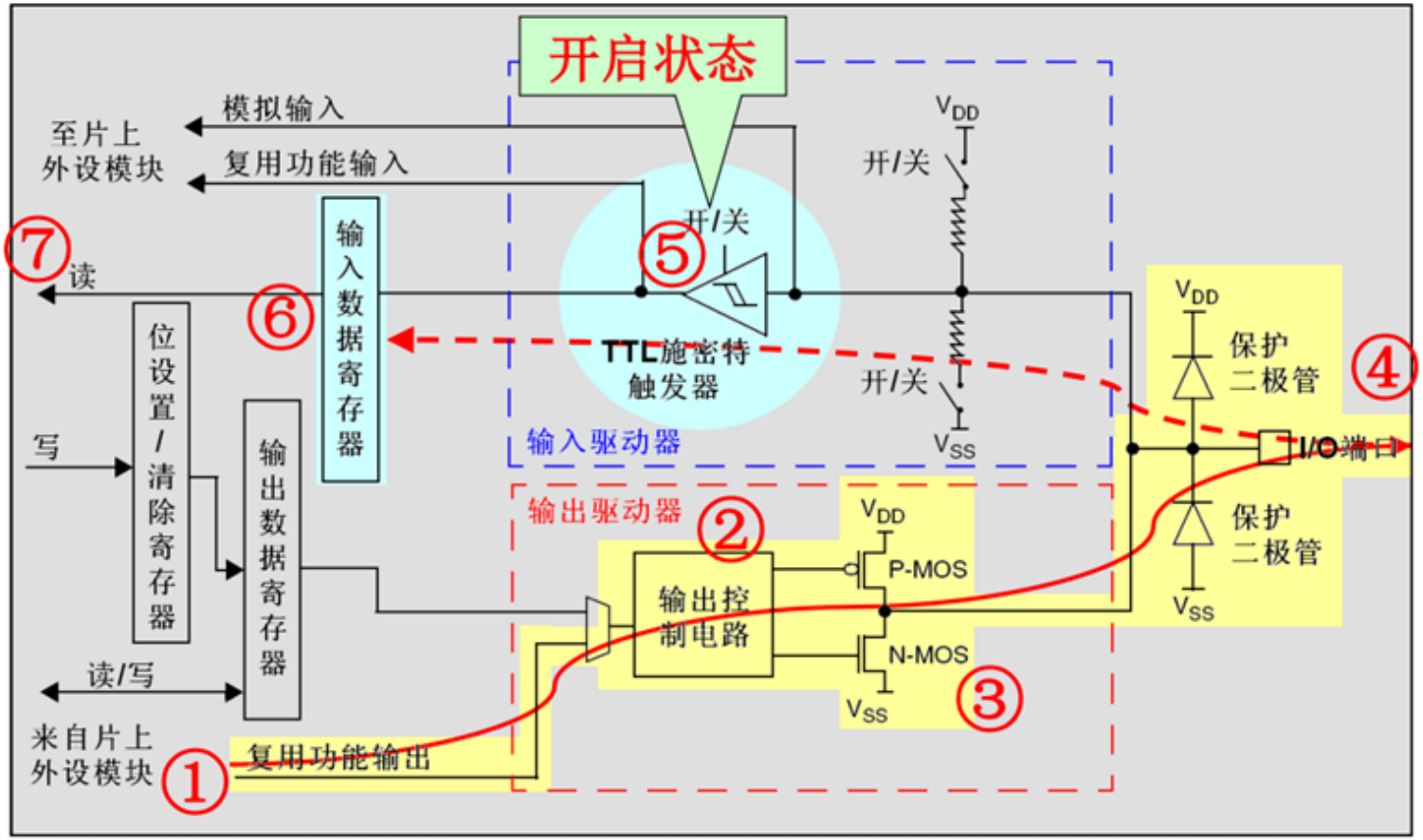
模拟输入模式


②
模拟输入
至片上
外设模块
复用功能输入
读
输入数据寄存器
0
开/关
TTL施密特
触发器
V_{DD}
开/关
开/关
V_{SS}
输入驱动器
V_{DD}
保护
二极管
①
I/O端口
保护
二极管
V_{SS}
写
位设置/清除寄存器
输出数据寄存器
输出驱动器
V_{DD}
P-MOS
输出控制电路
N-MOS
V_{SS}
读/写
来自片上
外设模块
复用功能输出

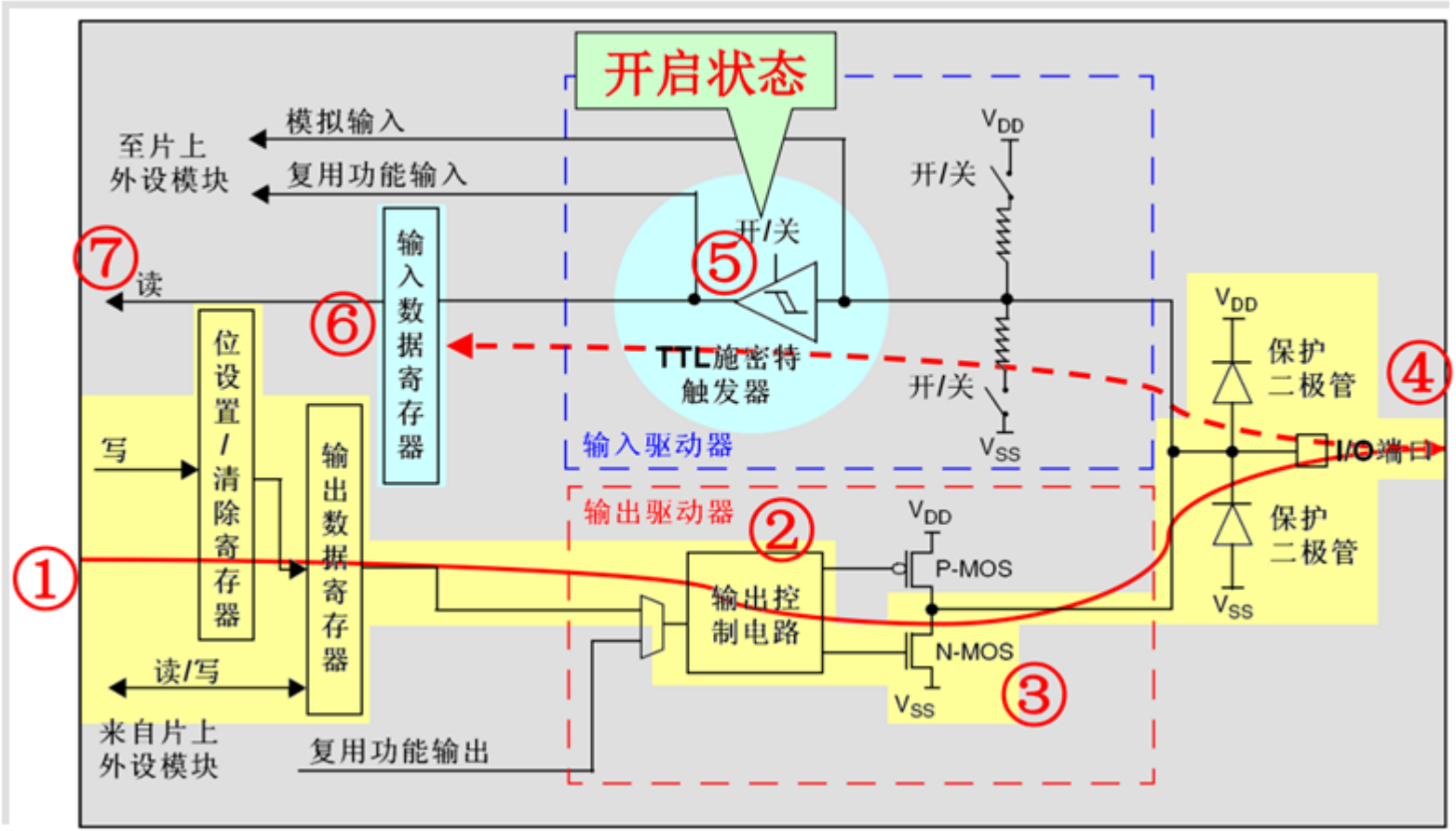
推挽输出模式


开启状态
模拟输入
至片上
外设模块
复用功能输入
⑦
读
⑥
输入数据寄存器
⑤
开/关
TTL施密特
触发器
V_{DD}
开/关
开/关
V_{SS}
输入驱动器
V_{DD}
保护
二极管
④
I/O端口
保护
二极管
V_{SS}
写
位设置/清除寄存器
输出数据寄存器
①
输出驱动器
②
输出控制电路
V_{DD}
P-MOS
N-MOS
V_{SS}
③
读/写
来自片上
外设模块
复用功能输出

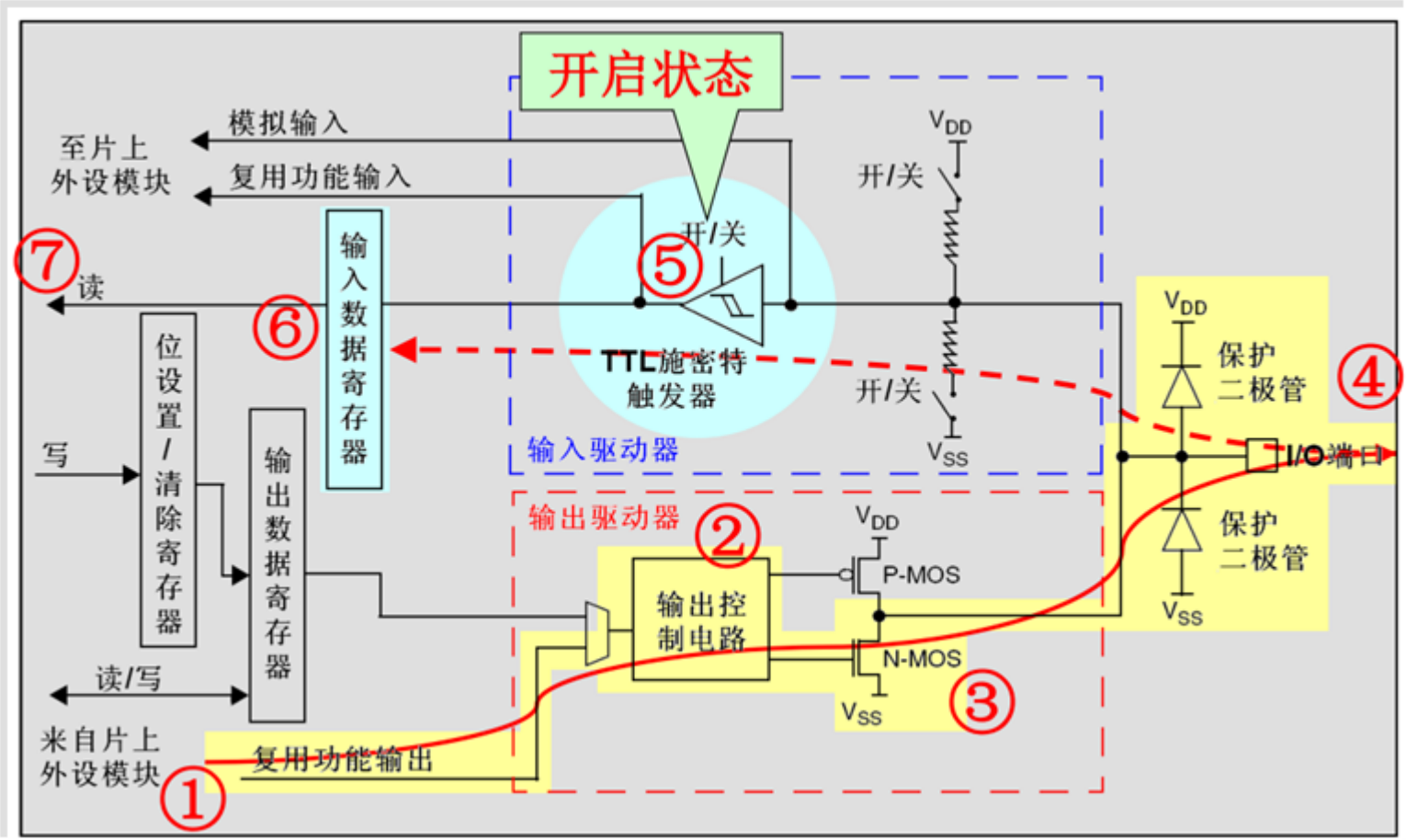
复用推挽输出模式


开启状态
模拟输入
至片上
外设模块
复用功能输入
⑦
读
⑥
输入数据寄存器
开/关
⑤
TTL施密特
触发器
V_{DD}
开/关
开/关
V_{SS}
输入驱动器
V_{DD}
保护
二极管
④
I/O端口
保护
二极管
V_{SS}
写
位设置/清除寄存器
输出数据寄存器
输出驱动器
②
V_{DD}
P-MOS
输出控制电路
N-MOS
V_{SS}
③
读/写
来自片上
外设模块
①
复用功能输出

开漏输出模式


开启状态
模拟输入
至片上
外设模块
复用功能输入
⑦
读
⑥
输入数据寄存器
⑤
开/关
TTL施密特
触发器
输入驱动器
V_{DD}
开/关
开/关
V_{SS}
V_{DD}
保护
二极管
④
I/O端口
保护
二极管
V_{SS}
写
位设置/清除寄存器
输出数据寄存器
①
读/写
来自片上
外设模块
复用功能输出
输出驱动器
②
V_{DD}
P-MOS
输出控制电路
N-MOS
V_{SS}
③

复用开漏输出模式


开启状态
模拟输入
至片上
外设模块
复用功能输入
⑦
读
⑥
输入数据寄存器
⑤
开/关
TTL施密特
触发器
V_{DD}
开/关
开/关
V_{SS}
输入驱动器
V_{DD}
保护
二极管
④
I/O端口
保护
二极管
V_{SS}
写
位设置/清除寄存器
输出数据寄存器
读/写
来自片上
外设模块
①
复用功能输出
输出驱动器
②
V_{DD}
P-MOS
输出控制电路
N-MOS
V_{SS}
③

输入模式

上拉输入	上拉输入模式，引脚内部有个上拉电阻，通过开关连接到电源VDD，当I/O引脚无输入信号时，**默认输入高电平。**
下拉输入	下拉输入模式和上拉输入模式正好相反，当I/O引脚无输入信号时，**默认输入低电平**。
浮空输入	浮空输入模式下引脚内部既不接上拉电阻也不连接下拉电阻，直接经施密特触发器输入I/O引脚的信号。
模拟输入	**引脚信号连接到芯片内部的片上外设**，其典型应用是A/D模拟输入，对外部信号进行采集。

输出模式

推挽输出	P-MOS管和N-MOS管轮流工作，可以输出高电平和低电平
复用推挽输出	与推挽输出模式很是类似。只是输出的高低电平的来源，不是让CPU直接写输出数据寄存器，取而代之利用片上外设模块的复用功能输出来决定的。
开漏输出	P-MOS管一直处于截止状态，只有N-MOS管工作，只能输出低电平。需要外接上拉电阻，才能实现输出高电平。
复用开漏输出	与开漏输出模式很是类似。只是输出的高低电平的来源，不是让CPU直接写输出数据寄存器，取而代之利用片上外设模块的复用功能输出来决定的。

03

GPIO模块的HAL库接口函数及应用

GPIO的HAL库接口函数

GPIO的 HAL库应用实例

GPIO的工程源码分析

GPIO的HAL库接口函数

Project

- Demo_GPIO
  - Application/MDK-ARM
  - Application/User
    - main.c
    - gpio.c
    - cmsis_armcc.h
    - cmsis_compiler.h
    - cmsis_version.h
    - core_cm3.h
    - gpio.h
    - main.h
    - stddef.h
    - stdint.h
    - stm32f103xe.h
    - stm32f1xx.h
    - stm32f1xx_hal.h
    - stm32f1xx_hal_conf.h
    - stm32f1xx_hal_cortex.h
    - stm32f1xx_hal_def.h
    - stm32f1xx_hal_dma.h
    - stm32f1xx_hal_dma_ex.h
    - stm32f1xx_hal_exti.h
    - stm32f1xx_hal_flash.h
    - stm32f1xx_hal_flash_ex.h
    - stm32f1xx_hal_gpio.h
    - stm32f1xx_hal_gpio_ex.h

main.c | gpio.c | stm32f1xx_hal_gpio.h

```
214
215 /* Exported functions ---------------------------------------------------------*/
216 /** @addtogroup GPIO_Exported_Functions
217   * @{
218   */
219
220 /** @addtogroup GPIO_Exported_Functions_Group1
221   * @{
222   */
223 /* Initialization and de-initialization functions ****************************/
224 void  HAL_GPIO_Init(GPIO_TypeDef  *GPIOx, GPIO_InitTypeDef *GPIO_Init);
225 void  HAL_GPIO_DeInit(GPIO_TypeDef  *GPIOx, uint32_t GPIO_Pin);
226 /**
227   * @}
228   */
229
230 /** @addtogroup GPIO_Exported_Functions_Group2
231   * @{
232   */
233 /* IO operation functions *****************************************************/
234 GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
235 void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState);
236 void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
237 HAL_StatusTypeDef HAL_GPIO_LockPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
238 void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin);
239 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin);
240
241 /**
242   * @}
243   */
244
245 /**
```

GPIO初始化与复位函数

GPIO引脚功能操作函数

GPIO的HAL库接口函数的源码在源文件stm32f1xx_hal_gpio.c中，其对应的头文件stm32f1xx_hal_gpio.h声明了GPIO所有的库函数，共8种。

GPIO的HAL库接口函数

类型	函数原型	功能描述
初始化及复位函数	HAL_GPIO_Init()	GPIO初始化函数
	HAL_GPIO_DeInit()	复位选定的端口引脚到初始状态
引脚功能操作函数	HAL_GPIO_ReadPin()	读取选定的端口引脚的电平状态
	HAL_GPIO_WritePin()	设置选定的端口引脚输出高电平或低电平
	HAL_GPIO_TogglePin()	设置选定端口引脚的电平状态翻转
	HAL_GPIO_LockPin()	当端口引脚电平状态改变时保持锁定时的值
	HAL_GPIO_EXTI_IRQHandler()	外部中断处理函数
	HAL_GPIO_EXTI_Callback()	中断回调函数

GPIO的HAL库接口函数

重点掌握

引脚初始化函数：HAL_GPIO_Init（）	
函数原型	void HAL_GPIO_Init（GPIO_TypeDef * GPIOx, GPIO_InitTypeDef * GPIO_Init）
功能描述	初始化指定引脚
入口参数1	GPIOx：端口号，取值范围GPIOA~GPIOD
入口参数2	GPIO_Init：指向引脚初始化数据类型GPIO_InitTypeDef的结构体指针，该结构体包含了指定引脚的配置参数
返回值	无
注意事项	该函数由CubeMX软件自动生成
参考示例	HAL_GPIO_Init（GPIOA, &gpio）

GPIO的HAL库接口函数

实例展示

```
GPIO_InitTypeDef  GPIO_InitStruct = {0}; // 定义引脚初始化变量，并初使化为0
// 配置引脚PA5：推挽输出，无上拉/下拉，输出速度为低速
 GPIO_InitStruct.Pin = GPIO_PIN_5|GPIO_PIN_10;
 GPIO_InitStruct.Mode = GPIOA_MODE_OUTPUT_PP;
 GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init（GPIOA, &GPIO_InitStruct）; // 调用引脚初始化函数完成引脚初始化
```

GPIO的HAL库接口函数

重点掌握

引脚读取函数：HAL_GPIO_ReadPin（）

函数原型	GPIO_PinState HAL_GPIO_ReadPin（GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin）
功能描述	读取指定引脚的电平状态
入口参数1	GPIOx：端口号，取值范围为GPIOA~GPIOG
入口参数2	GPIO_Pin：引脚号，取值范围为GPIO_PIN_0~GPIO_PIN_15
返回值	GPIO_PinState:表示引脚电平状态的枚举类型变量，当取值为： GPIO_PIN_SET:表示读到高电平GPIO_PIN_RESET:表示读到低电平
注意事项	该函数由用户调用
参考示例	HAL_GPIO_ReadPin（GPIOA, GPIO_PIN_4）

GPIO的HAL库接口函数

函数源码（了解）

源码解析：

读取输入数据寄存器IDR的值与指定引脚进行按位与操作，若结果不为0，则返回高电平；若结果为0，则返回低电平。

GPIOx为引脚的端口号，取值为GPIOA~GPIOG，表示指向GPIO结构体的指针，用于访问端口引脚的寄存器。

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx,
uint16_t GPIO_Pin)
{
  GPIO_PinState bitstatus;

  if ((GPIOx->IDR & GPIO_Pin) != (uint32_t)GPIO_PIN_RESET)
  {
      bitstatus = GPIO_PIN_SET;
  }
  else
  {
      bitstatus = GPIO_PIN_RESET;
  }
  return bitstatus;
}
```

GPIO的HAL库接口函数

重点掌握

引脚写入函数：HAL_GPIO_WritePin（）	
函数原型	void HAL_GPIO_WritePin（GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState ）
功能描述	设置指定引脚输出高电平或低电平
入口参数1	GPIOx：端口号，取值范围为GPIOA~GPIOG
入口参数2	GPIO_Pin：引脚号，取值范围为GPIO_PIN_0~GPIO_PIN_15
入口参数3	PinState:表示引脚电平状态的枚举类型变量，取值为GPIO_PIN_SET，表示输出高电平；取值为GPIO_PIN_SETGPIO_PIN_RESET，表示输出低电平
返回值	无
注意事项	该函数由用户调用
参考示例	HAL_GPIO_WritePin（GPIOA, GPIO_PIN_12, GPIO_PIN_RESET）

GPIO的HAL库接口函数

函数源码

```
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx,
uint16_t GPIO_Pin, GPIO_PinState PinState)
{
  if (PinState != GPIO_PIN_RESET)
  {
    GPIOx->BSRR = GPIO_Pin;
  }
  else
  {
    GPIOx->BSRR = (uint32_t)GPIO_Pin << 16U;
  }
}
```

源码解析（了解）

将指定引脚的电平状态写入置位/复位寄存器BSRR中，该寄存器为32位，其中高16位控制端口16个引脚（0~15）输出低电平，低16位控制端口16个引脚输出高电平。

GPIOx_BSRR寄存器的高16位称为清除寄存器，低16位称为置位寄存器。

GPIO的HAL库接口函数

重点掌握

引脚翻转函数：HAL_GPIO_TogglePin（）	
函数原型	void HAL_GPIO_TogglePin（GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin）
功能描述	翻转指定引脚的电平状态
入口参数1	GPIOx：端口号，取值范围为GPIOA~GPIOG
入口参数2	GPIO_Pin：引脚号，取值范围为GPIO_PIN_0~GPIO_PIN_15
返回值	无
注意事项	该函数由用户调用
参考示例	HAL_GPIO_TogglePin（GPIOB, GPIO_PIN_15）

GPIO的HAL库接口函数

函数源码（了解）

```
void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx,
uint16_t GPIO_Pin)
{
        if ((GPIOx->ODR & GPIO_Pin) != 0x00u)
        {
                GPIOx->BRR = (uint32_t)GPIO_Pin;
        }
        else
        {
                GPIOx->BSRR = (uint32_t)GPIO_Pin;
        }
}
```

源码解析：

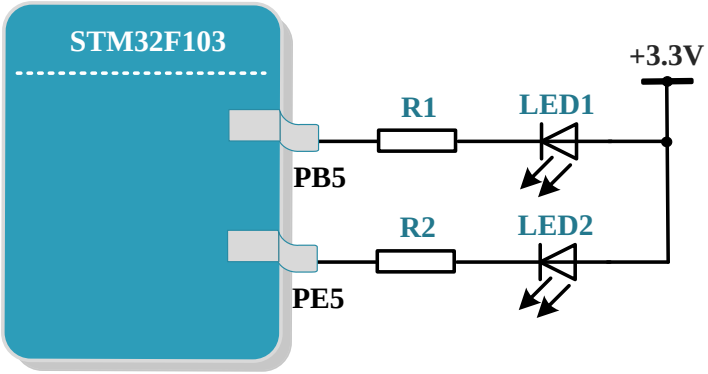
读出输出数据寄存器ODR的值与指定引脚进行按位与操作，结果若为真，表明原来为高电平，则写入BRR寄存器（BRR寄存器写入“1”有效，写入“0”不影响ODR的状态）；若结果为假，表明原来为低电平，则控制BSRR寄存器低16位部分做置位（置“1”）操作。

GPIO的HAL库应用实例

功能

采用基于HAL库设计方式，利用两个GPIO引脚输出高低电平控制发光二极管，并按一定时间间隔改变IO口电平，实现灯光闪烁效果。

硬件设计

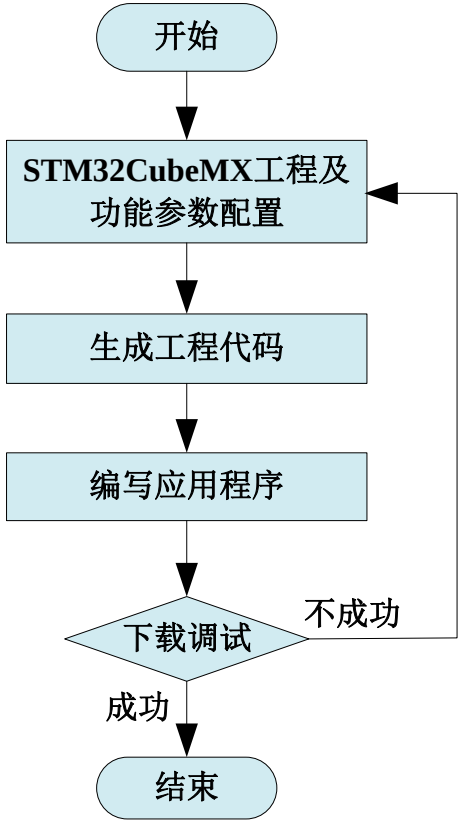


当引脚输出为低电平时，LED灯亮，当引脚输出为高电平时，LED灯灭。

GPIO的HAL库应用实例

基于HAL库的软件设计流程如图所示：

- 建立STM32CubeMX工程；
- 进行功能参数配置；
- 生成工程代码;
- 修改代码完成应用程序设计;
- 下载到开发板测试。



设计步骤——新建STM32CubeMX工程，选择设计采用的MCU

在D盘或其他盘符目录下新建一个文件夹，用来存放后面建立的STM32CubeMX工程。

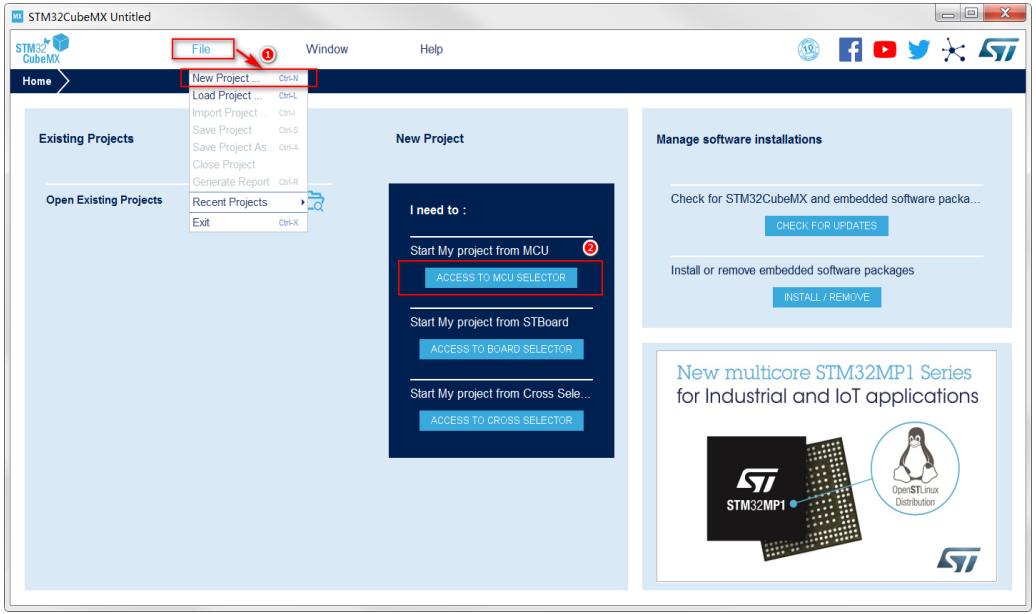
注意

所建工程文件名必须是英文名称，且必须是英文路径。

设计步骤——新建STM32CubeMX工程，选择设计采用的MCU

打开STM32CubeMX软件

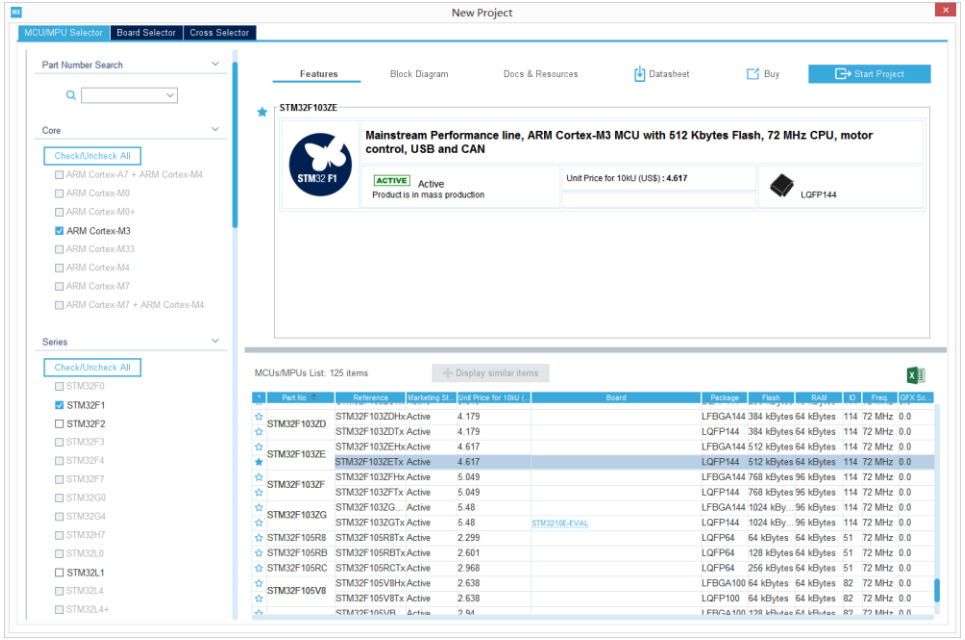
STM32CubeMX



在主界面中通过单击“New Project”下“ACCESS TO MCU SELECTOR”按钮或通过菜单栏中的“File”→“New Project”新建一个工程

设计步骤——新建STM32CubeMX工程，选择设计采用的MCU

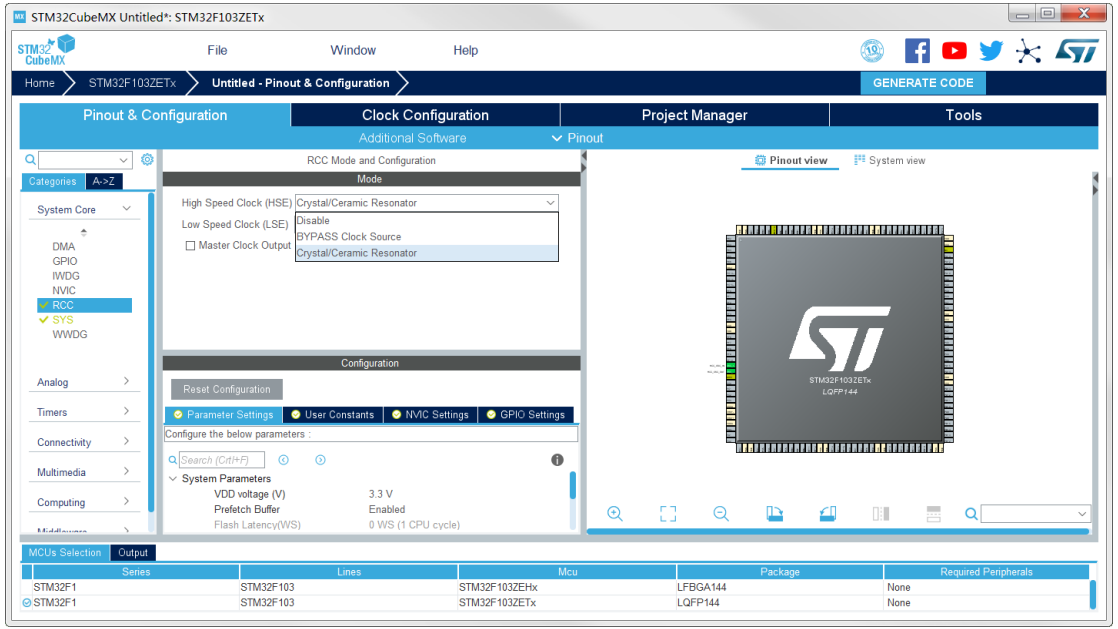
- 选择所用MCU对应的内核，选择“ARM Cortex-M3”；
- 在“Series”选项中选择对应的系列，如“SMT32F1”；
- 在右侧“MCUs/MPUs List”中找到该系列下所使用的微控制器芯片型号
- 选中并单击对应芯片型号，在“MCUs/MPUs List”上方会显示该芯片对应的基本信息。



也可以通过在搜索栏内直接输入芯片型号采用搜索的方式来快速完成。

设计步骤——STM32CubeMX功能参数配置（ RCC配置）

在“Categories”栏目中的“System Core”中，找到“RCC”选项

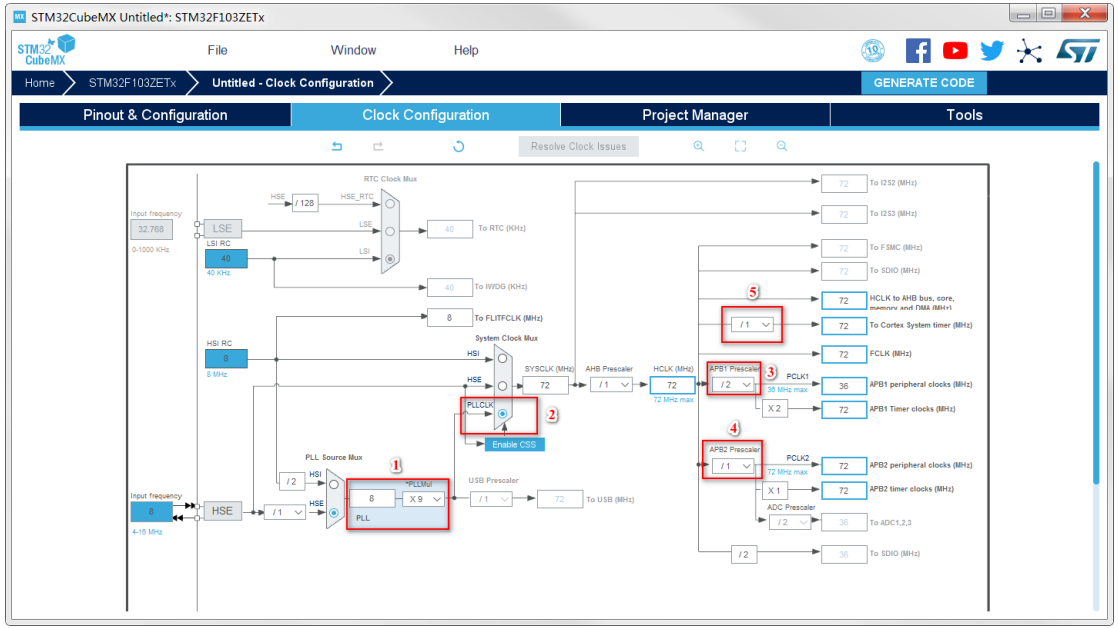


时钟信号选择HSE作为系统的外部时钟源，HSE选择“Crystal/Ceramic Resonator”（晶振/陶瓷谐振器）

LSE选择“Disable”

设计步骤——STM32CubeMX功能参数配置（时钟配置）

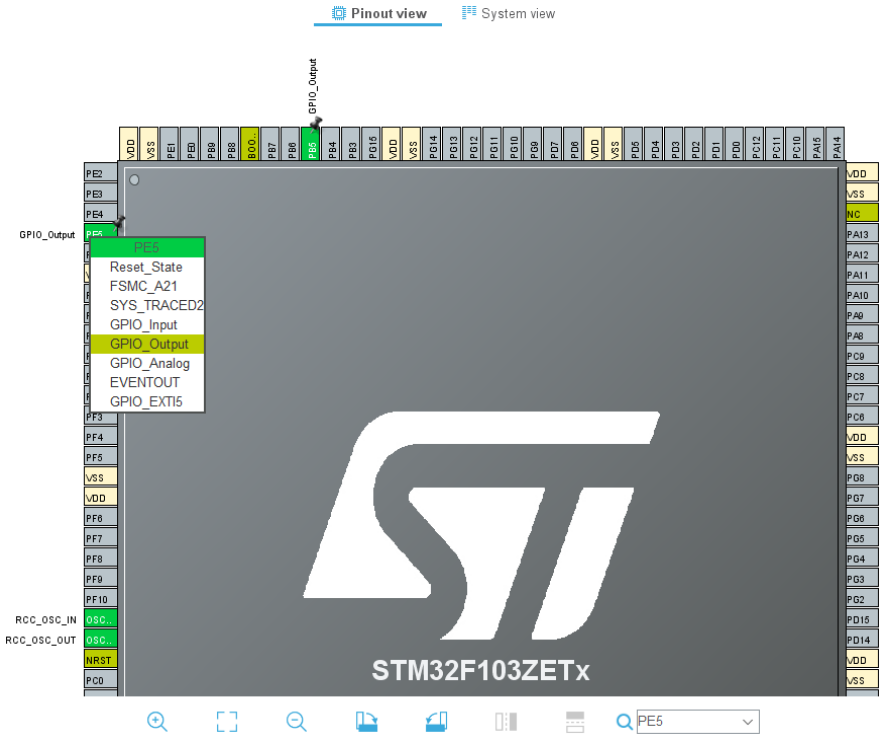
点开“Clock Configuration”选项栏，进行系统时钟配置



- 采用HSE外部晶振，频率为8MHz，通过PLL的9倍频，使得系统时钟SYSCLK为72MHz；
- APB2时钟与HCLK相同，所以不需要分频；
- HCLK经2分频得到APB1的时钟频率为PCLK1=36MHz，

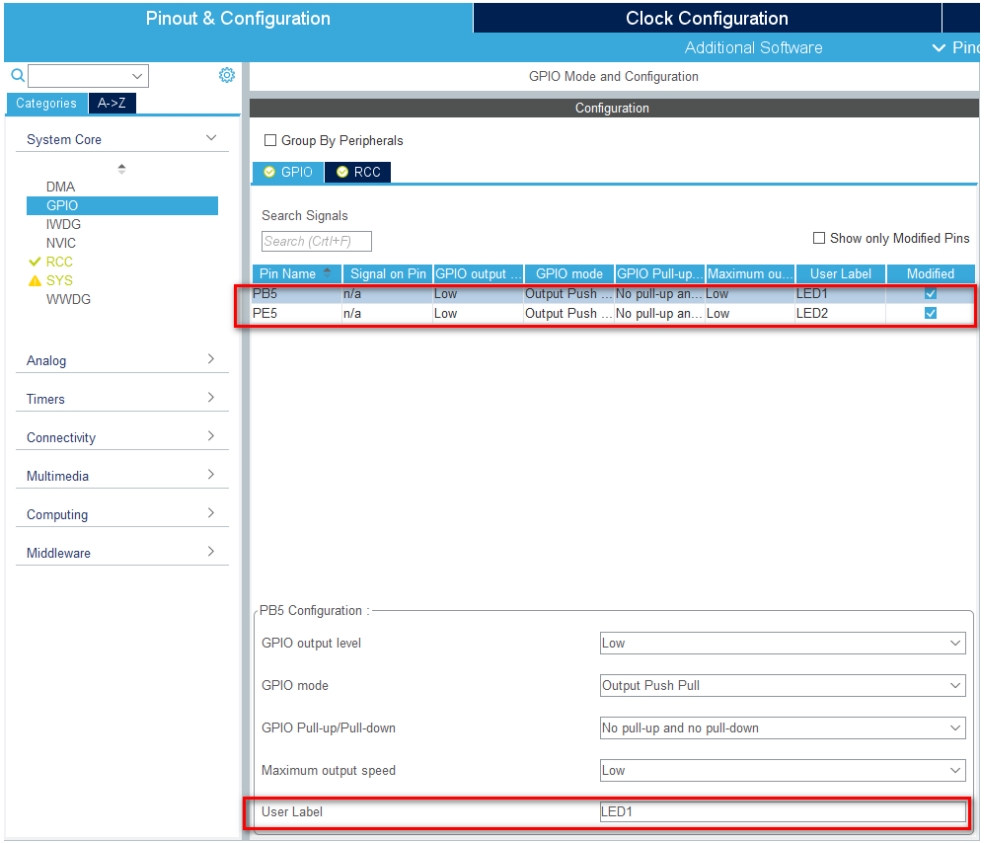
设计步骤——STM32CubeMX功能参数配置（MCU引脚选择）

在STM32CubeMX的主界面的左侧“Categories”栏目中的“System Core”中，找到“GPIO”选项，在STM32CubeMX的主界面的右侧“Pinout view”芯片引脚图中选中~~PE5、PB5~~引脚，设置为GPIO_Output选项。



学会看开发板原理图，看一下开发板的LED灯连接在了哪个GPIO口？

设计步骤——STM32CubeMX功能参数配置（GPIO引脚参数配置）



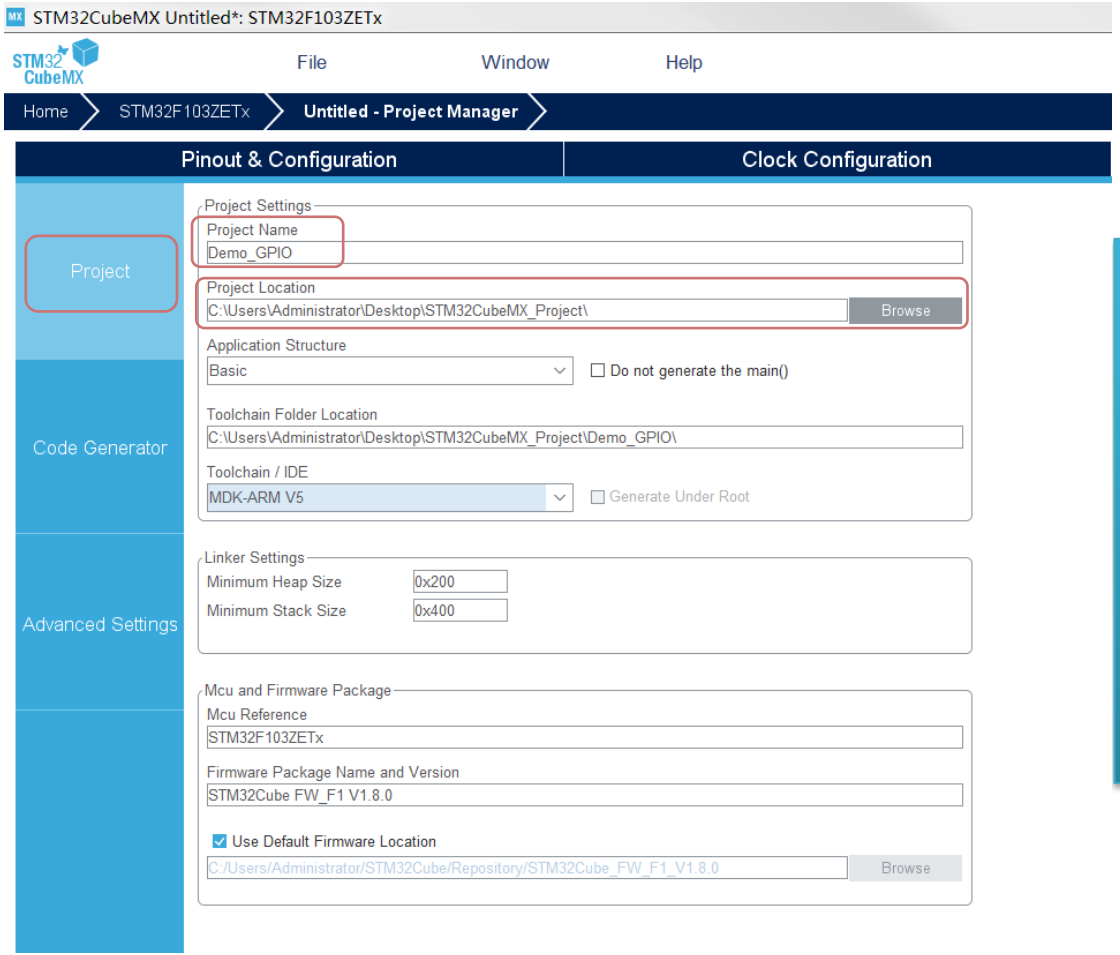
选择好引脚后，在左侧“Configuration”栏目下显示所选择的引脚，单击，对应显示该引脚的参数设置表，设置相应参数，如输出电平、模式、用户标签等。

设计步骤——生成工程代码

点击STM32CubeMX的主界面中的“Project Manager”菜单，在弹出的页面中单击“Project”项，输入项目名称“Demo_GPIO”，选择存放路径。

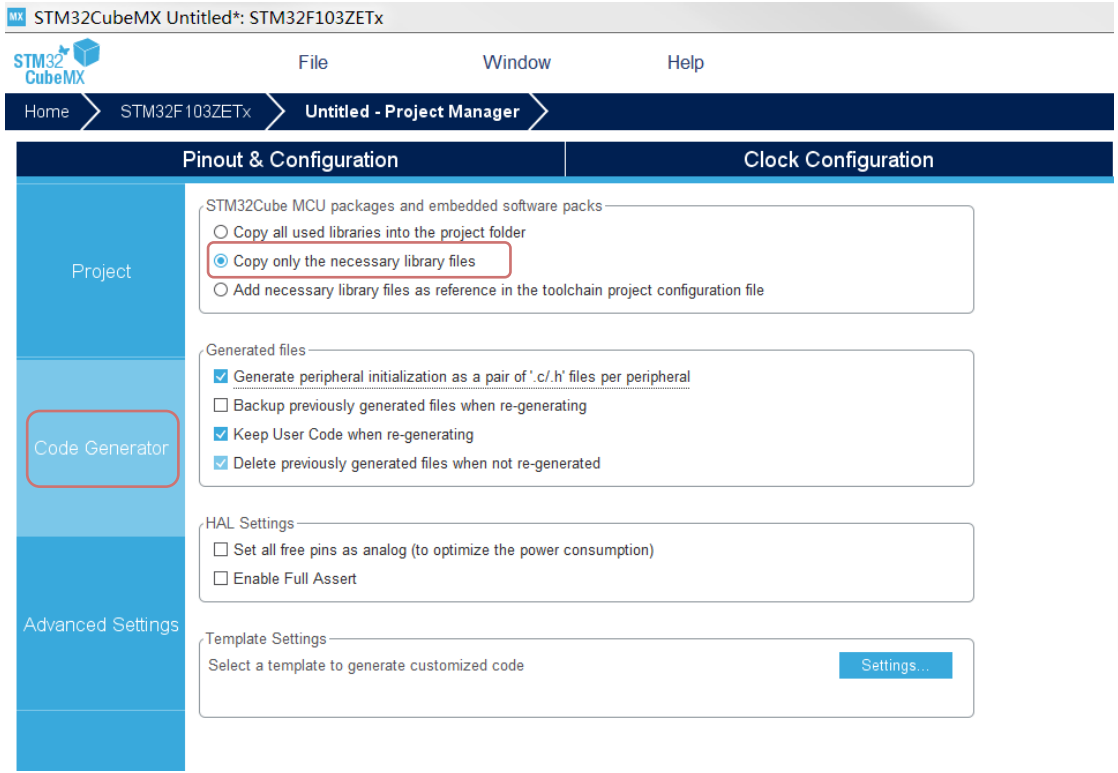
说明

用户可以自己定义，项目名称一般应反映项目内容，便于管理

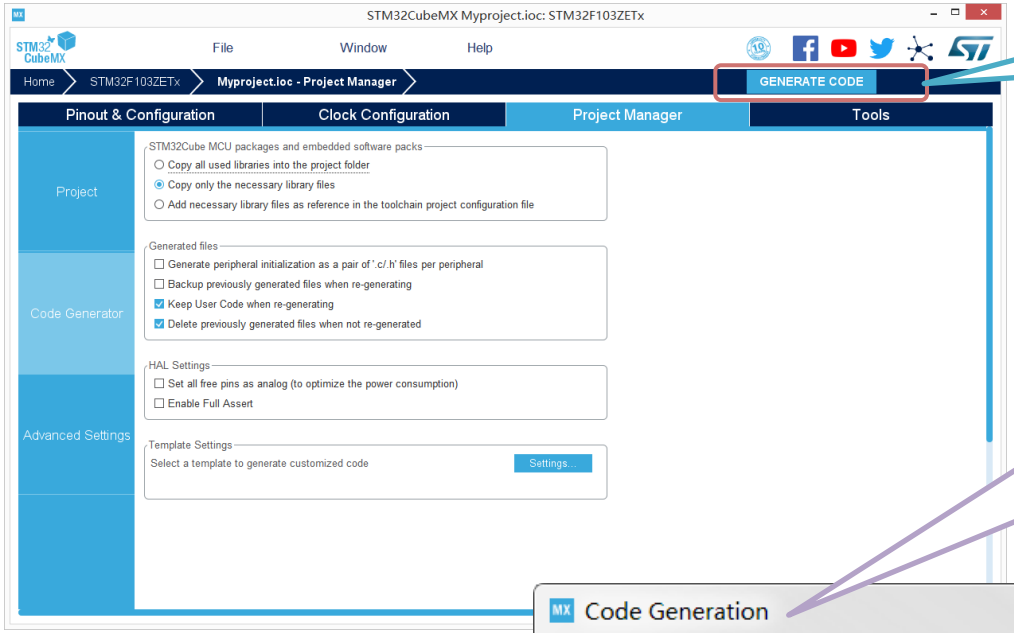


设计步骤——生成工程代码

- 点击“Code Generator”选项
- 在“STM32Cube MCU packages and embedded software packs”栏内选择第二个单选项“Copy only the necessary library files”，仅拷贝必须的库文件；
- 在“Generated files”栏内复选第1、3、4选项

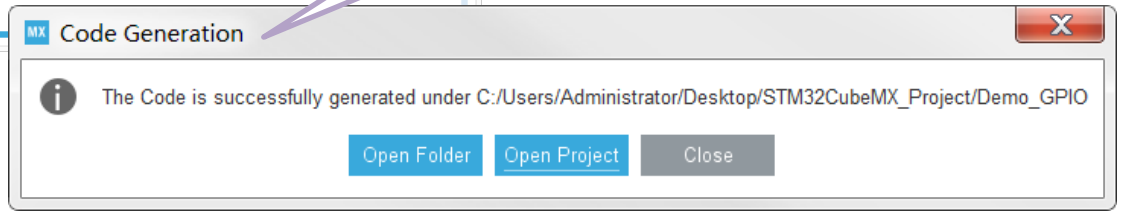


设计步骤——生成工程代码



单击右上角“GENERATE CODE”按钮，即可生成对应工程代码。

成功后弹出Code Generation窗口，提示代码生成成功，用户可以根据下一步需要选择打开文件夹、打开项目或关闭。



设计步骤——编写应用程序

使用STM32CubeMX生成的工程，其用户功能代码即应用程序代码的编写有位置规范要求。

本实例的用户应用程序代码写在/* USER CODE BEGIN 3 */ 和 /* USER CODE END 3 */ 之间：

```
while (1)
{
  /* USER CODE END WHILE */
  /* USER CODE BEGIN 3 */
    HAL_GPIO_TogglePin(LED1_GPIO_Port,LED1_Pin);
          //LED1---PB5状态翻转
    HAL_Delay(100); //延时100毫秒
    HAL_GPIO_TogglePin(LED2_GPIO_Port,LED2_Pin);
         //LED2---PE5状态翻转
    HAL_Delay(100); //延时100毫秒
}
```

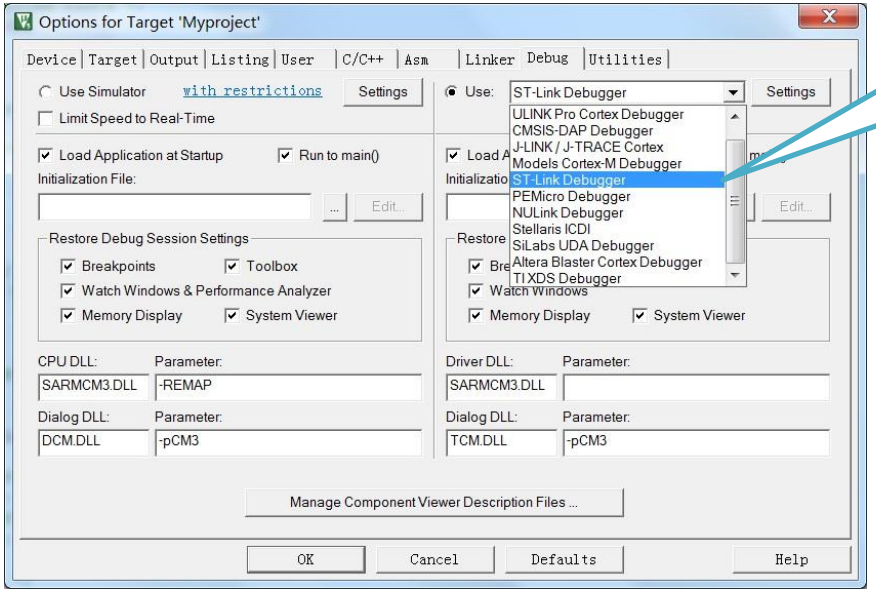
说明

用户自己编写的应用程序需写在**/* USER CODE BEGIN x */** 和 **/* USER CODE END x */** 之间。

注意

写在其他地方，在使用STM32CubeMX重新配置和生成工程时，会删除该代码。

设计步骤——下载调试验证



在“Debug”选项卡中，根据所使用的开发板下载调试工具，选择相应的选项，这里选择ST-Link调试下载器

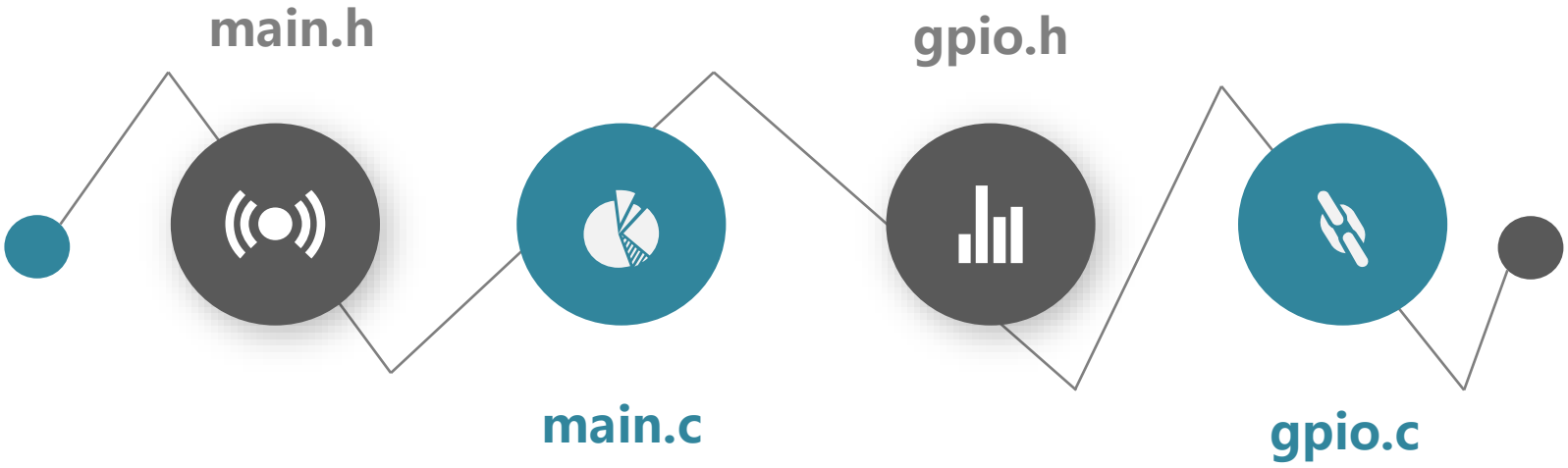
单击“Build(F7)”编译按钮，进行重新编译。

将开发板通过USB线与PC机进行连接

通过工具栏里的“Download”按钮将编译通过的程序下载到开发板上

通过观察开发板上相应LED灯的闪烁情况来验证程序的正确性。

GPIO工程源码解析



针对GPIO-LED闪烁工程相关源码共四个文件

GPIO工程源码解析

main.c

```
    /* Includes 头文件*/
#include "main.h"
#include "gpio.h"

    /* 函数声明 */
void SystemClock_Config(void); //设置系统时钟

int main(void)
{
        /* 将所有的外设复位，
           并初始化Flash和系统滴答时钟Systick */
 HAL_Init();
      /*配置系统时钟*/
 SystemClock_Config();
      /***GPIO初始化，函数实现定义在gpio.c文件中
       *此代码对应于STM32CubeMX软件中的Pinout &
       Configuration设置的引脚参数
      */
 MX_GPIO_Init();
```

```
while (1)
 {
      //以下为用户编写的应用程序代码

       /*调用HAL_GPIO_TogglePin函数
          使LED1---PB5状态翻转 */

HAL_GPIO_TogglePin(LED1_GPIO_Port,LED1_Pin);
HAL_Delay(100);
      //调用HAL库函数HAL_Delay延时100毫秒
      /*调用HAL_GPIO_TogglePin函数
         使LED2---PE5状态翻转 */
  HAL_GPIO_TogglePin(LED2_GPIO_Port,LED2_Pin);
  HAL_Delay(100);
       //调用HAL库函数HAL_Delay延时100毫秒
 }
}
```

GPIO工程源码解析

main.c

```
    /***系统时钟配置，对应于STM32CubeMX软件中的系统时钟操作*/
void SystemClock_Config(void)
{
        /*OSC外部晶振初始化结构体变量，用于打开HSE，设置PLL等 */
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
        /*SYSCLK初始化结构体变量，用于选择sysclk的时钟源、设置AHB预分频系数*/
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
       /* 给RCC_OscInitStruct的成员赋值，用于选择系统时钟源，设置PLL、PLLMul的值等 */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;//HSE为外部晶振8MHz
  RCC_OscInitStruct.HSEState = RCC_HSE_ON; //打开HSE
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;//设置HSE预分频系数为1
  RCC_OscInitStruct.HSIState = RCC_HSI_ON; //打开HSI（高速内部时钟，HSI=8MHz）
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;//打开PLL
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;//PLL的输入时钟选择为HSE
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;//设置PLLMul为9，倍频到72MHz
```

GPIO工程源码解析

main.c

```
    /***调用HAL_RCC_OscConfig()函数对系统时钟进行初始化配置， 通过if语句判断是否成功，
        如果初始化配置不成功，则由Error_Handler()进行处理*/
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
 {    Error_Handler();//出错处理函数    }
    /*给RCC_ClkInitStruct的成员赋值，用于选择SYSCLK的时钟源， 设置AHB、APB1、APB2的时钟，
    此代码对应于STM32CubeMX软件中的Clock Configuration配置的系统时钟操作 */
 RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK|RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
        /*将 SYSCLK的时钟源选为PLLCLK */
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
        /* 将AHB Prescaler的分频值设为 1，AHB时钟频率=72MHz*/
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
        /* 将APB1 Prescaler的分频值设为2，APB2时钟频率=PCLK1 = 36 MHz*/
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
        /*将APB2 Prescaler的分频值设为1，APB2时钟频率=PCLK2 = 72MHz */
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
  /*判断SYSCLK时钟初始化配置是否成功， 如果初始化配置不成功，则由Error_Handler()进行处理*/
if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {   Error_Handler();    }
}
```

GPIO工程源码解析

main.c

```
/* 错误处理函数 */
void Error_Handler(void)
{
            //错误处理代码由用户编写
}


/* 断言函数，如果使用断言的话，需定义宏USE_FULL_ASSERT，这里没有使用断言*/
#ifdef  USE_FULL_ASSERT
void assert_failed(uint8_t *file, uint32_t line)
{
    //断言失败函数，具体处理代码由用户编写
}
#endif /* USE_FULL_ASSERT */
```

GPIO工程源码解析

gpio.c

```
  /* Includes 头文件*/
#include "gpio.h"

/***GPIO初始化，方式和标准外设库的GPIO_Init类似
*此代码对应于STM32CubeMX软件中的Pinout & Configuration设置的
引脚参数*/

void MX_GPIO_Init(void)
{
 GPIO_InitTypeDef GPIO_InitStruct = {0};

     /* 使能GPIOB、GPIOE端口时钟 */
 __HAL_RCC_GPIOE_CLK_ENABLE();
 __HAL_RCC_GPIOB_CLK_ENABLE();

    /* 配置GPIO初始引脚LED2-PE5为低电平 */
 HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin, GPIO_PIN_RESET);

   /* 配置GPIO初始引脚LED1-PB5为低电平 */
 HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
```

```
     /* LED2-PE5引脚参数配置 */
 GPIO_InitStruct.Pin = LED2_Pin; //I/O引脚设置为LED2-PE5
 GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    //引脚模式设置为推挽输出
 GPIO_InitStruct.Pull = GPIO_NOPULL; //无上拉
 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
     //输出速度为低速
 HAL_GPIO_Init(LED2_GPIO_Port, &GPIO_InitStruct);
    //调用初始化函数初始化PE5

     /* LED1-PB5引脚参数配置 */
 GPIO_InitStruct.Pin = LED1_Pin; //I/O引脚设置为LED1-PB5
 GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
     //引脚模式设置为推挽输出
 GPIO_InitStruct.Pull = GPIO_NOPULL; //无上拉
 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
     //输出速度为低速
 HAL_GPIO_Init(LED1_GPIO_Port, &GPIO_InitStruct);
     //调用初始化函数初始化PB5
}
```

GPIO工程源码解析

main.h

```
/*条件编译，防止头文件被重复包含*/
#ifndef __MAIN_H
#define __MAIN_H

#ifdef __cplusplus
extern "C" {
#endif

/* Includes 头文件,stm32f1xx_hal.h这个头文件很重要*/
#include "stm32f1xx_hal.h"
```

```
/* 函数声明 */
void Error_Handler(void);
/* 端口引脚的宏定义，方便修改 */
#define LED2_Pin GPIO_PIN_5
#define LED2_GPIO_Port GPIOE
#define LED1_Pin GPIO_PIN_5
#define LED1_GPIO_Port GPIOB

#ifdef __cplusplus
}
#endif

#endif /* __MAIN_H */
```

GPIO工程源码解析

gpio.h

```
    /* 条件编译，防止头文件被重复包含*/
#ifndef __gpio_H
#define __gpio_H
#ifdef __cplusplus
extern "C" {
#endif
    /* Includes 头文件*/
#include "main.h"
    /* 函数声明 */
void MX_GPIO_Init(void);
#ifdef __cplusplus
}
#endif
#endif /*__ pinoutConfig_H */
```

基于HAL库开发的一般流程

开启系统时钟

- 在STM32CubeMX主界面中的“Pinout & Configuration”选项页将RCC设置系统外部时钟源为HSE，HSE选择“Crystal/Ceramic Resonator”（晶振/陶瓷谐振器）

配置系统时钟树

- 将外部的8MHz晶振经PLL倍频9倍得到72MHz的系统时钟，然后设置相应外设时钟，如APB1为系统时钟的2分频为36MHz，APB2设置为与系统时钟相同

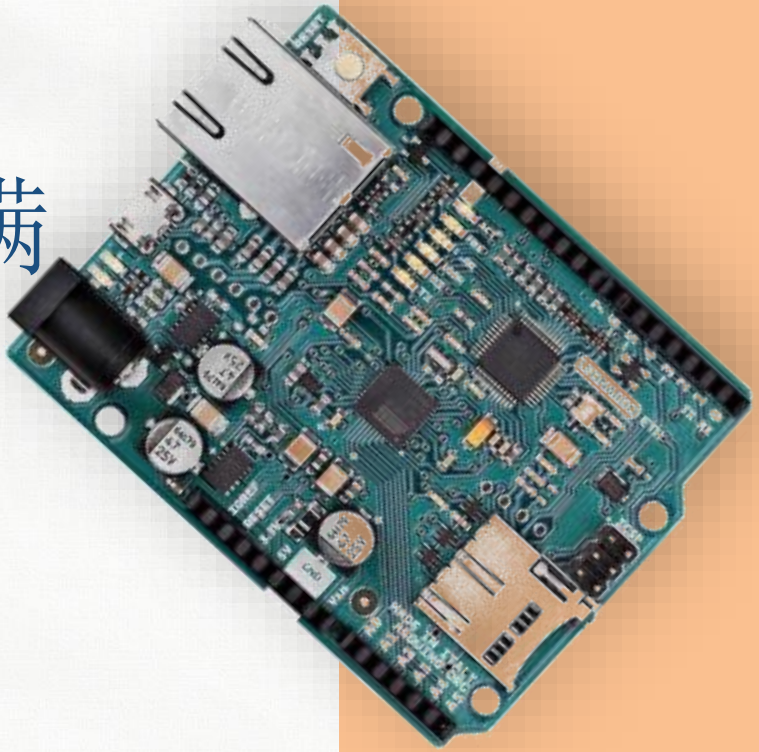
外设参数配置

- 在STM32CubeMX主界面中的“Pinout & Configuration”选项页对片上外设进行功能配置，如GPIO、ADC、定时器、NVIC等

生成工程代码

- 设置好工程所要存放的位置及所使用的集成开发环境，就可以生成工程代码

应用程序编写

- STM32CubeMX配置完成后，生成工程代码，就可以在Keil软件中编写应用程序，以实现相应功能。

本章结束，收获满满